



ACCUKNOX

Secure Code to Cognition™

ASPM Playbook

Cut through 10,000 findings. Fix the five that can actually bring you down.

Certified & Accredited by



As Featured In



Available On



Table Of Contents

i) Pre-Requisites

Label and Token Creation

1 Container Scan

GitHub Actions Integration

Jenkins Integration

View Findings

Use Cases

2 IaC Scan

GitHub Actions Integration

Jenkins Integration

View Findings

Use Cases

3 SAST Scan

Github Actions Integration

Jenkins Integration

Findings Page

Use Cases

4 DAST Scan

GitHub Actions Integration

Jenkins Integration

Authenticated/Non Authenticated Scan using Collectors

View Findings

Use Cases

5 Secret Scanning

Github Actions Integration

Jenkins Integration

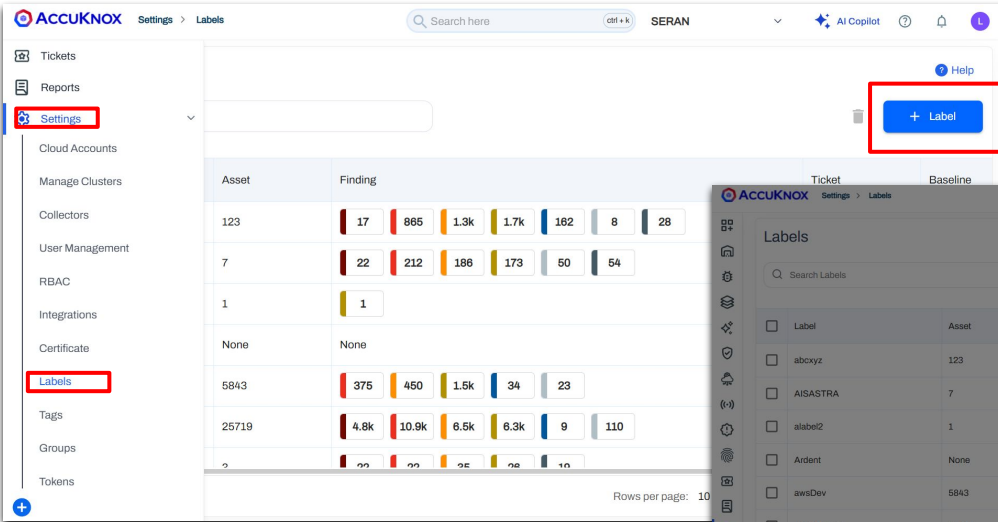
Findings Page

Use Cases

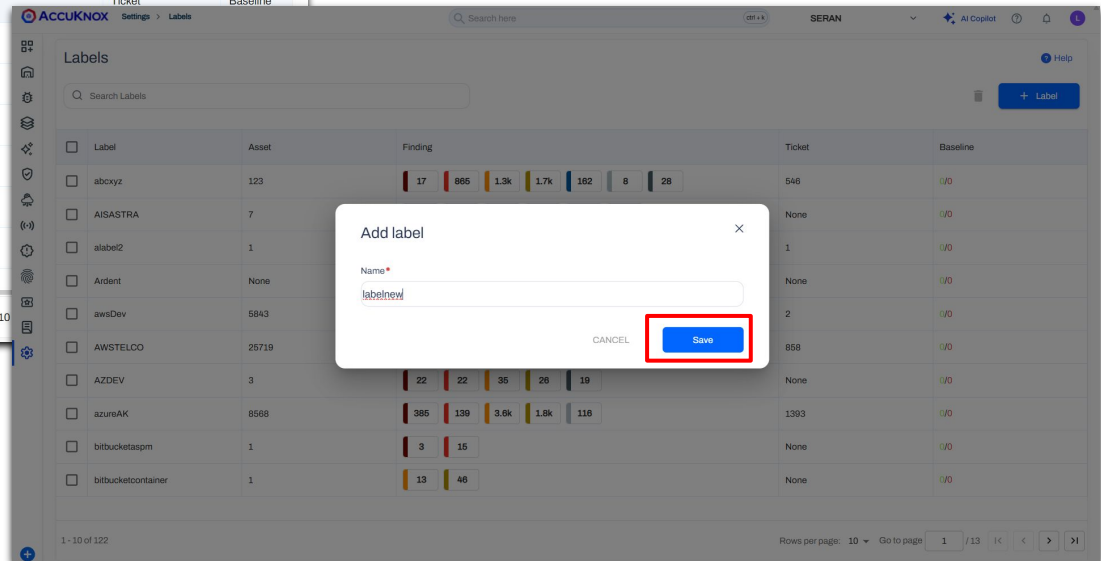
[Click here to explore the Help Docs for full documentation for various modules.](#)

Generate AccuKnox Label for CI/CD pipeline

- To generate a label, open AccuKnox and navigate to **Settings > Labels > Create**.
- Copy the label, then configure them as secrets in your CI/CD pipeline.



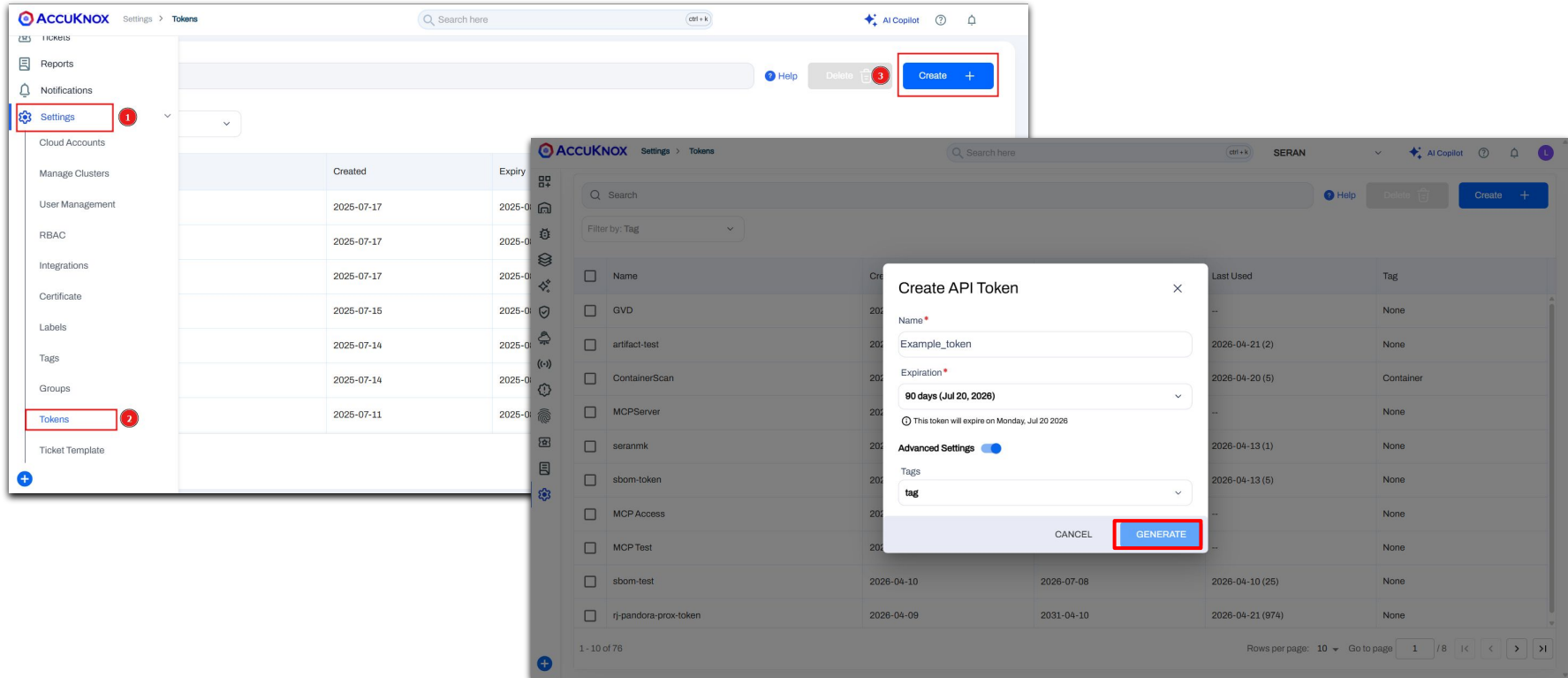
The screenshot shows the AccuKnox interface with the 'Settings > Labels' path selected. The left sidebar has 'Settings' highlighted. The main area shows a table of labels with columns for Asset, Finding, Ticket, and Baseline. The '+ Label' button is highlighted in the top right corner.



The screenshot shows the AccuKnox 'Labels' page. An 'Add label' dialog box is open, showing a form to create a new label. The 'Name' field is filled with 'labelnew'. The 'Save' button is highlighted in the bottom right corner of the dialog box.

Generate AccuKnox API token for CI/CD pipeline

- To generate a token, open AccuKnox and navigate to **Settings > Tokens > Create**.
- Generate a token, and copy the token, then configure them as secrets in your CI/CD



The image displays two screenshots of the AccuKnox web interface. The left screenshot shows the 'Settings' menu on the left sidebar, with 'Tokens' highlighted. The right screenshot shows the 'Tokens' page, where a 'Create' button is highlighted. A modal window titled 'Create API Token' is open, showing fields for 'Name' (Example_token), 'Expiration' (90 days), and 'Tags' (tag). The 'GENERATE' button is highlighted.

AccuKnox Settings > Tokens

Search here [ctrl + k]

Help Delete [3] Create +

Filter by: Tag

Create API Token

Name *
Example_token

Expiration *
90 days (Jul 20, 2026)
This token will expire on Monday, Jul 20 2026

Advanced Settings ☒

Tags
tag

CANCEL GENERATE

Name	Created	Expiry	Last Used	Tag
GVD	2025-07-17	2025-07-17	2026-04-21 (2)	None
artifact-test	2025-07-17	2025-07-17	2026-04-20 (5)	Container
ContainerScan	2025-07-17	2025-07-17	2026-04-20 (5)	None
MCPServer	2025-07-15	2025-07-15	2026-04-13 (1)	None
seranmk	2025-07-14	2025-07-14	2026-04-13 (5)	None
sbom-token	2025-07-14	2025-07-14	2026-04-13 (5)	None
MCP Access	2025-07-11	2025-07-11	2026-04-13 (5)	None
MCP Test	2025-07-11	2025-07-11	2026-04-13 (5)	None
sbom-test	2025-07-11	2025-07-11	2026-04-13 (5)	None
rj-pandora-prox-token	2025-07-11	2025-07-11	2026-04-13 (5)	None

1 - 10 of 76 Rows per page: 10 Go to page 1 / 8



Container Scan

ASPM Integration

Configuring the Container Scan in GitHub actions



Step 1: Add [AccuKnox Container Scan](#) to Your GitHub Workflow

- Open your GitHub repository and navigate to your workflow file (typically .github/workflows/your-workflow.yml).
- After the build step, add the AccuKnox container scan GitHub Action.
- Copy the label and token created, then configure them as secrets in your CI/CD along with Accuknox Endpoint.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select Container Image Findings.
- Click on any finding that interests you to view detailed information and recommendations.

[Visit Help Docs for complete guidance](#)

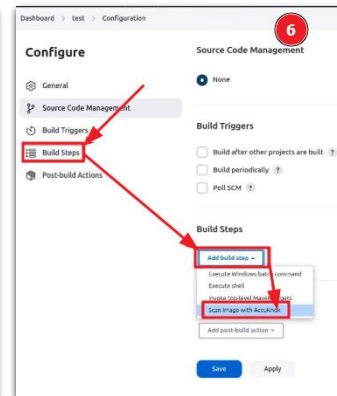
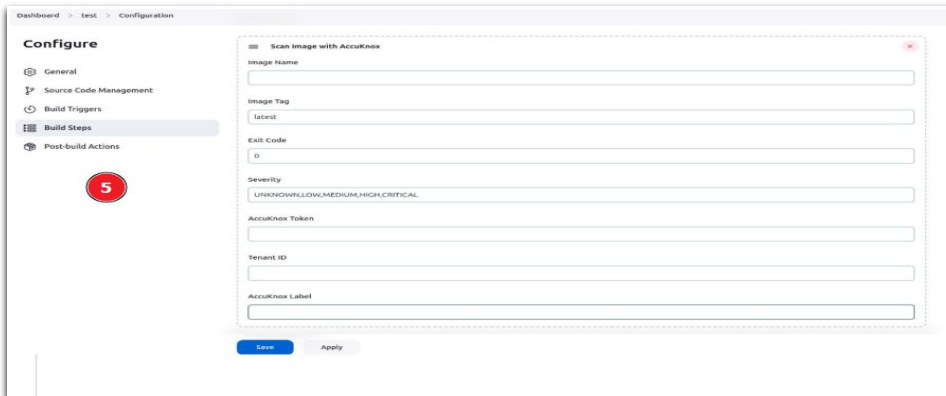
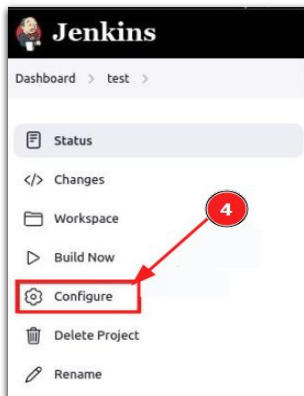
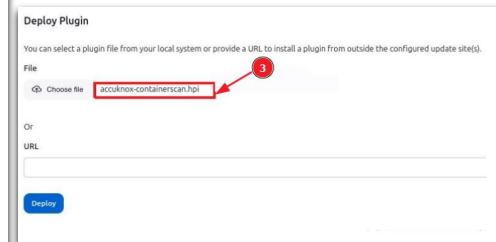
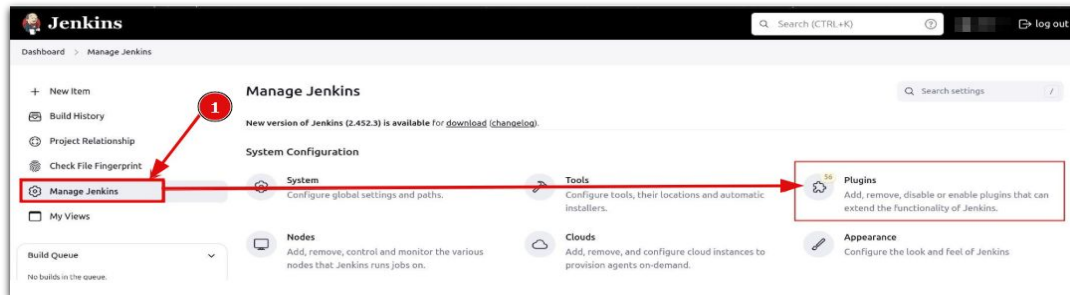
```
jobs:
  accuknox-cicd:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@main

      - name: Build Docker image
        run: |
          docker buildx build .

      - name: AccuKnox Container Scan
        uses: accuknox/container-scan-action@v0.0.1
        with:
          token: ${ secrets.TOKEN }
          tenant_id: ${ secrets.TENANT_ID }
```

Configuring the Container Scan in Jenkins

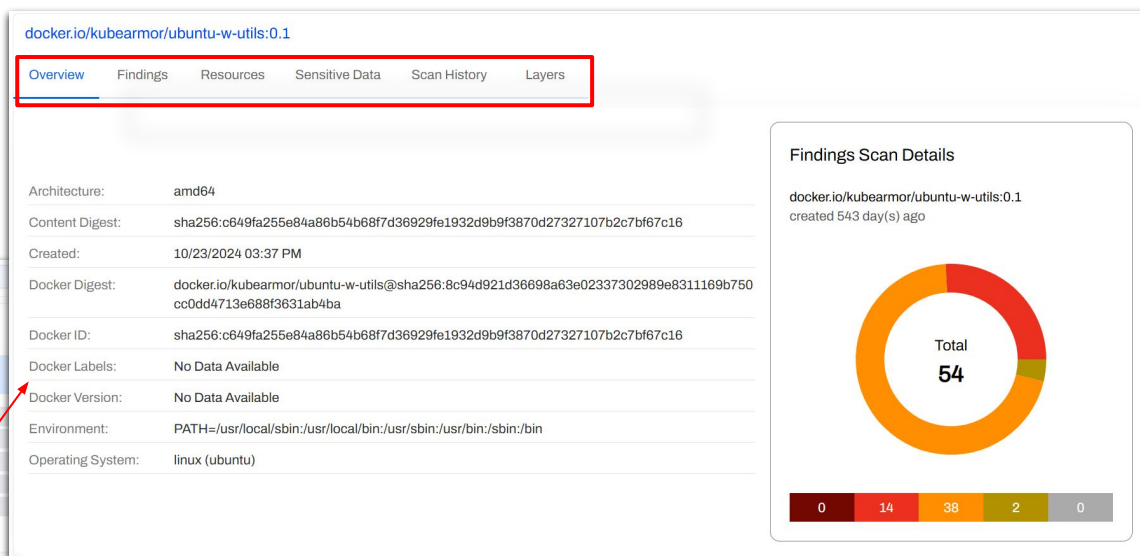
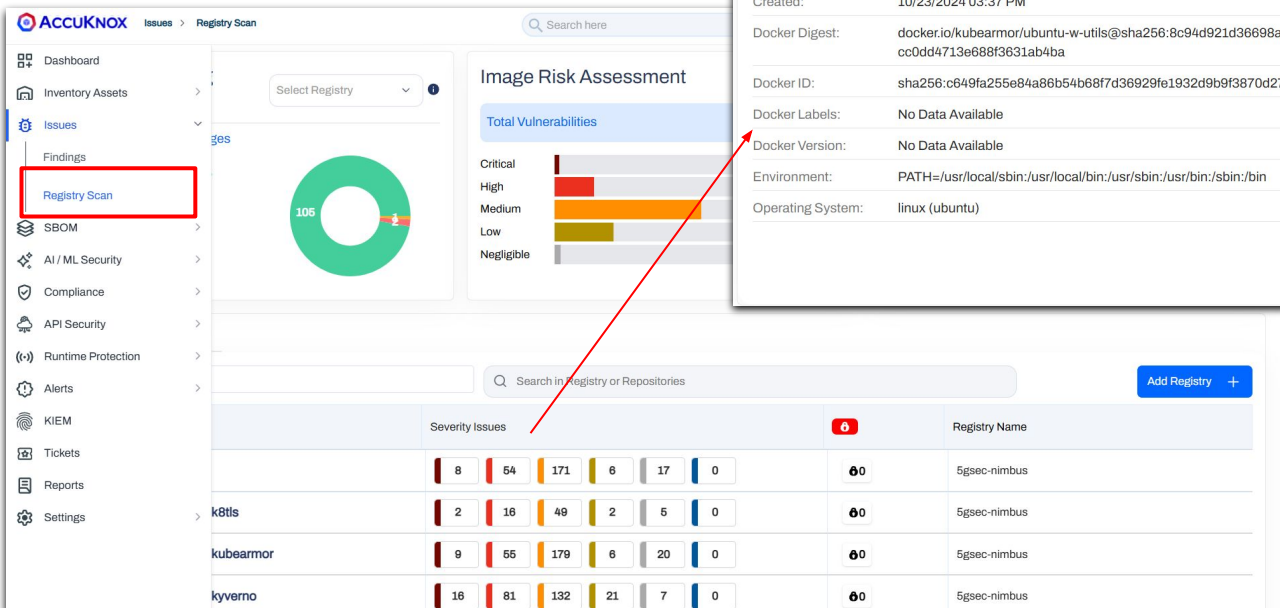
- Manage Jenkins > Plugins > Advanced settings > Upload & install
- Open your Jenkins job, add build step, select **Scan image with AccuKnox**, and fill required details



[Visit Help Docs for complete guidance](#)

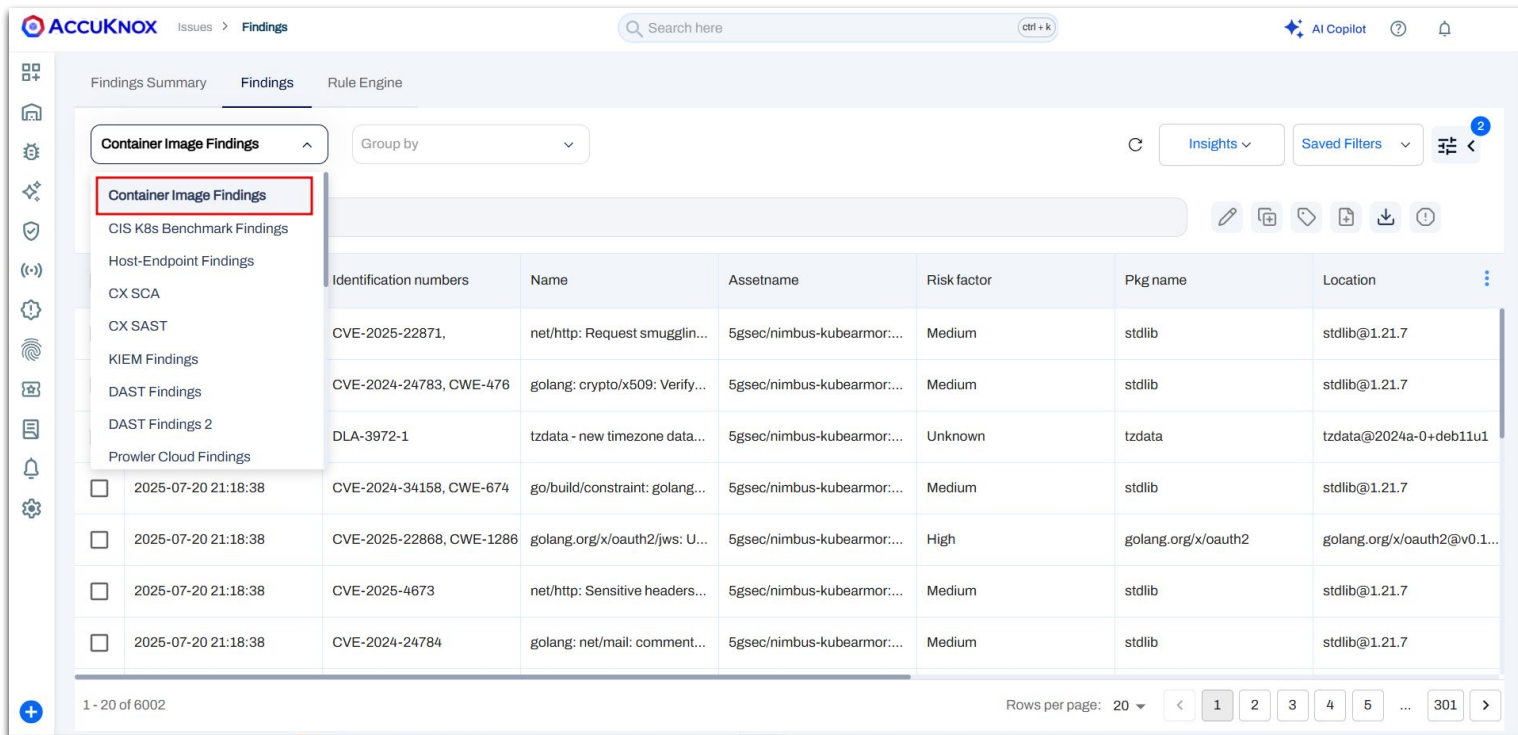
View Findings in Registry Scan Page

- You can see the findings on the **Registry scan** page.
- Click any of the findings to get more details.



View Findings in Findings page

- Alternatively, you can view the findings on the **Findings** page. Select the **Container Image Findings** to access the relevant details.



The screenshot displays the AccuKnox Findings page. The left sidebar contains a list of finding categories, with 'Container Image Findings' highlighted by a red box. The main content area shows a table of findings. The table has columns for Identification numbers, Name, Assetname, Risk factor, Pkg name, and Location. The findings are listed in a table with 6 columns: Identification numbers, Name, Assetname, Risk factor, Pkg name, and Location. The findings are listed in a table with 6 columns: Identification numbers, Name, Assetname, Risk factor, Pkg name, and Location.

Identification numbers	Name	Assetname	Risk factor	Pkg name	Location
CVE-2025-22871,	net/http: Request smugglin...	5gsec/nimbus-kubearmor...	Medium	stdlib	stdlib@1.21.7
CVE-2024-24783, CWE-476	golang: crypto/x509: Verify...	5gsec/nimbus-kubearmor...	Medium	stdlib	stdlib@1.21.7
DLA-3972-1	tzdata - new timezone data...	5gsec/nimbus-kubearmor...	Unknown	tzdata	tzdata@2024a-0+deb11u1
2025-07-20 21:18:38 CVE-2024-34158, CWE-674	go/build/constraint: golang...	5gsec/nimbus-kubearmor...	Medium	stdlib	stdlib@1.21.7
2025-07-20 21:18:38 CVE-2025-22868, CWE-1286	golang.org/x/oauth2/jws: U...	5gsec/nimbus-kubearmor...	High	golang.org/x/oauth2	golang.org/x/oauth2@v0.1...
2025-07-20 21:18:38 CVE-2025-4673	net/http: Sensitive headers...	5gsec/nimbus-kubearmor...	Medium	stdlib	stdlib@1.21.7
2025-07-20 21:18:38 CVE-2024-24784	golang: net/mail: comment...	5gsec/nimbus-kubearmor...	Medium	stdlib	stdlib@1.21.7

Use Case 1

Dependency analysis - Scanning for supply chain vulnerabilities

Use Case 2

Scan for sensitive data exposure

Use Case 3

Authentication Vulnerabilities

Use Case 1: Dependency analysis - Scanning for Supply chain vulnerabilities

- AccuKnox Container Scan identified a vulnerability in **Golang version 1.14.8**.
- The vulnerability is classified as **Cross-Site Scripting (XSS)**.
- AccuKnox provides **actionable remediation guidance and solution** to resolve the issue.

golang: default Content-Type setting in net/http/cgi and net/http/cgi could cause XSS: (libgcc@4.8.5-39.el7), CVE-2020-24553 Medium [🔗](#) [✕](#)

Age	Severity	SLA	CVE ID	Tickets Created
1 day	● Medium	60 days	CVE-2020-24553	0

[Description](#) [Solution](#) [Classification](#) [Other Info](#) [Raw Information](#)

Go before 1.14.8 and 1.15.x before 1.15.1 allows XSS because text/html is the default for CGI/FCGI handlers that lack a Content-Type header.

Finding for in resource Container | Container

First detected on 01/12/2025

Last detected on 02/12/2025

Compliance Frameworks
No compliance found

[Create Ticket](#) [Ask AI](#)

Asset
docker.elastic.co/beats/filebeat:7.5.0

Asset Type
Container

Status
● **Active**

Ignored
☐ No

Location
libgcc@4.8.5-39.el7

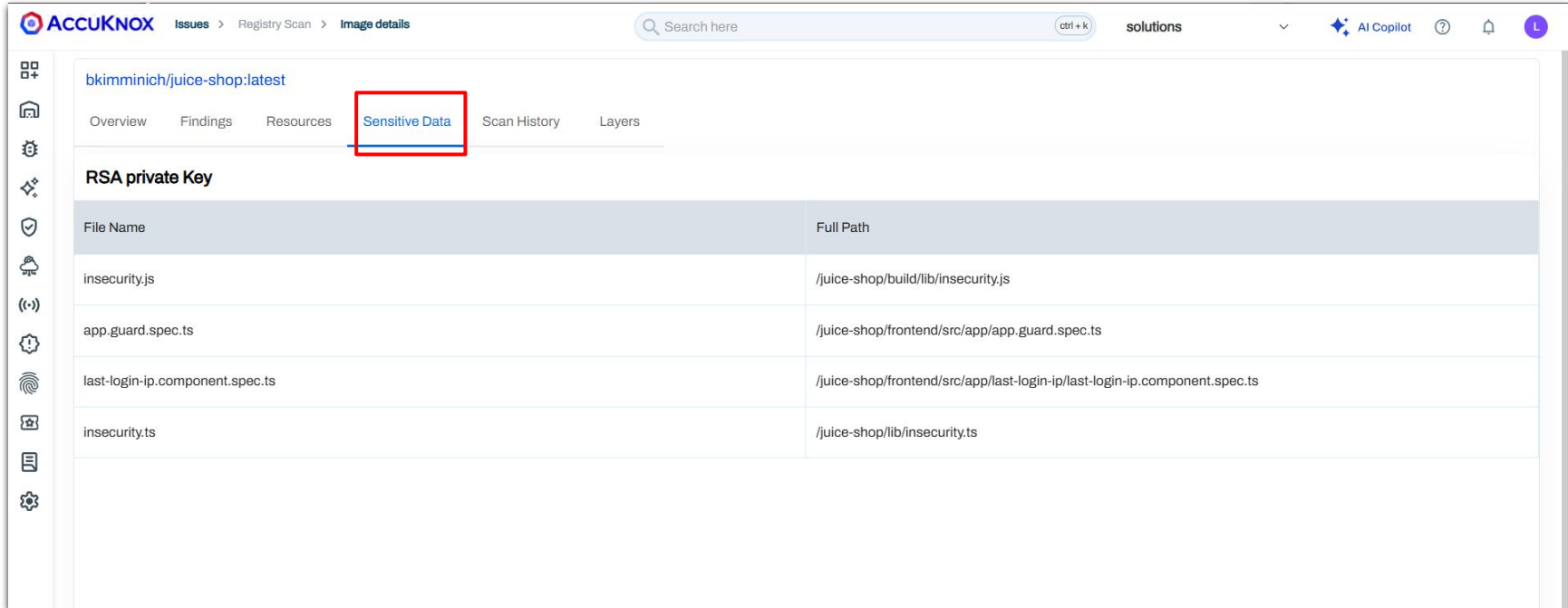
Ask AI [🔗](#) [✕](#)

To address the identified vulnerability (CVE-2020-24553) in the container image `docker.elastic.co/beats/filebeat:7.5.0`, follow these specific remediation steps:

- Identify the Affected Package:**
 - The vulnerability pertains to the Go programming language's `net/http/cgi` and `net/http/cgi` packages, which have issues with default Content-Type settings potentially leading to XSS attacks. This is indirectly related to the `libgcc` package version `4.8.5-39.el7` mentioned in the report.
- Update the Go Version:**
 - Since the vulnerability affects Go versions before 1.14.8 and 1.15.x before 1.15.1, the primary remediation is to update the Go runtime to a secure version. Ensure your application is built with Go version 1.14.8 or later, or 1.15.1 or later.
- Rebuild the Container Image:**
 - If you are directly using Go in your application within the container, ensure the application is recompiled using the updated Go version.
 - If the base image or any dependencies are using vulnerable Go versions, update these to ensure compatibility and security. Check the Dockerfile for any `FROM` statements that might be pulling a vulnerable Go version and update them accordingly.

Use Case 2: Scan for Sensitive Data Exposure

- The container image contains multiple exposed **RSA private keys**
- This represents a critical case of **sensitive data exposure**
- AccuKnox Container Scan detects such sensitive information within images



The screenshot displays the AccuKnox web interface. The top navigation bar includes the AccuKnox logo, a breadcrumb trail (Issues > Registry Scan > Image details), a search bar, and a 'solutions' dropdown. The left sidebar contains various icons for navigation. The main content area shows the image 'bkimminich/juice-shop:latest' with tabs for Overview, Findings, Resources, Sensitive Data (highlighted with a red box), Scan History, and Layers. Under the 'Sensitive Data' tab, a section titled 'RSA private Key' is visible. Below this, a table lists the detected sensitive data:

File Name	Full Path
insecurity.js	/juice-shop/build/lib/insecurity.js
app.guard.spec.ts	/juice-shop/frontend/src/app/app.guard.spec.ts
last-login-ip.component.spec.ts	/juice-shop/frontend/src/app/last-login-ip/last-login-ip.component.spec.ts
insecurity.ts	/juice-shop/lib/insecurity.ts

Use Case 3: Authentication Vulnerabilities

- AccuKnox identified an **authentication bypass vulnerability** in the container image
- The issue is associated with **curl/libcurl (version 7.88.1)**
- This flaw may allow attackers to **bypass authentication mechanisms** under certain conditions
- Proposed solution: Upgrade **curl/libcurl to a patched version** (latest stable release)

curl: curl: Authentication bypass due to incorrect connection reuse with Negotiate authentication: Medium  

(curl@7.88.1-10+deb12u5), CVE-2026-1965

Age	Severity 	SLA	CVE ID	Tickets Created
34 days	● Medium	60 days	CVE-2026-1965	0

Description Solution Classification Other Info Raw Information

libcurl can in some circumstances reuse the wrong connection when asked to do an Negotiate-authenticated HTTP or HTTPS request.

libcurl features a pool of recent connections so that subsequent requests can reuse an existing connection to avoid overhead.
[Show More](#)

 Finding for in resource Container | Container

 **First detected** on 15/03/2026

 **Last detected** on 17/04/2026

 Create Ticket

 Ask AI

Asset

quay.io/accuknox/myrepo:v11

Asset Type

Container

Status

● Active

Ignored

☐ No

Location

curl@7.88.1-10+deb12u5

 Ask AI  

To remediate the vulnerability identified in the curl package within the container image `quay.io/accuknox/myrepo:v11`, follow these steps:

1. Identify the Affected Package and Version:

- The affected package is `curl`.
- The installed version with the vulnerability is `7.88.1-10+deb12u5`.

2. Check for Available Fix:

- As of the provided information, there is no specific fixed version mentioned. However, it's essential to check the official sources for any updates that might have been released after this information was recorded.

3. Upgrade the Package:

- If a fixed version is available, you can upgrade the package using a package manager. For a Debian-based system, you can run:

```
bash  Copy code
```

```
apt-get update  
apt-get install --only-upgrade curl
```

- Ensure that the version you are upgrading to is greater than `7.88.1-10+deb12u5` and addresses CVE-2026-1965.

4. Rebuild the Container Image:

- Once the package is upgraded, rebuild your container image to ensure the changes are included:

```
bash  Copy code
```

```
docker build -t quay.io/accuknox/myrepo:v11 .
```




laC Scan

ASPM Integration

Configuring the IAC scan in GitHub actions

Step 1: Add [AccuKnox IAC scan GitHub action](#) to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically `github/workflows/your-workflow.yml`).
- After the build step, add the AccuKnox IaC scan GitHub Action.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select IaC Findings.
- Click on any finding that interests you to view detailed information and recommendations.

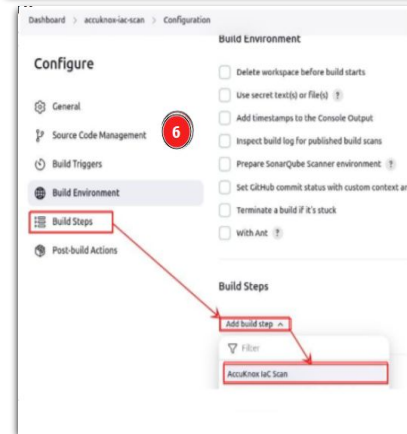
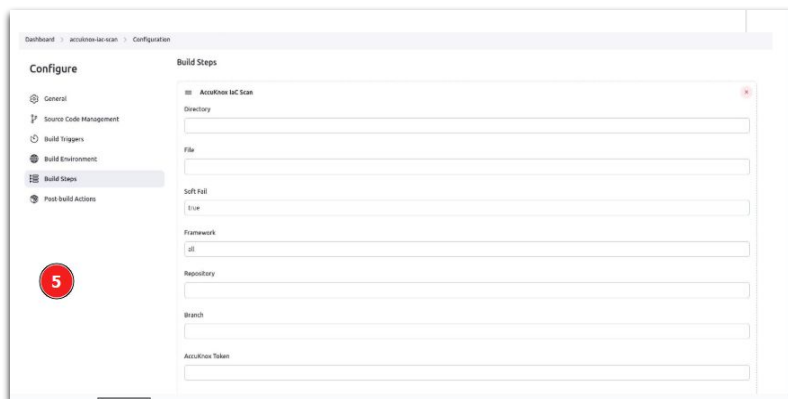
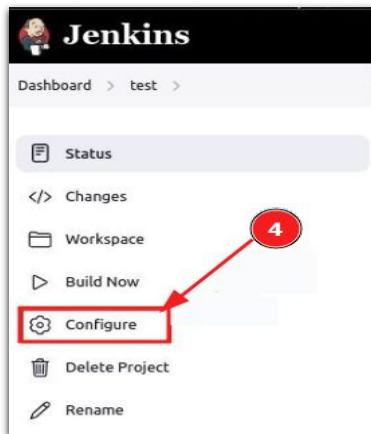
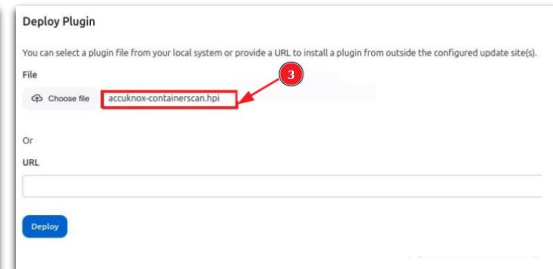
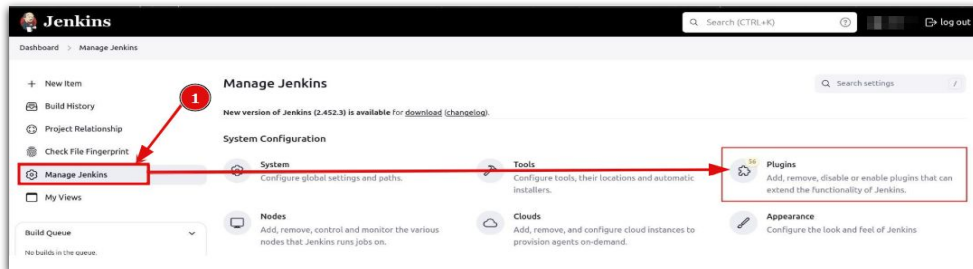
[Visit Help Docs for complete guidance](#)

```
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@main

      - name: Run IaC scan
        uses: accuknox/iac-scan-action@v0.0.1
        with:
          directory: ./
          output_file_path: ./results
          token: ${{ secrets.TOKEN }}
          endpoint: ${{ secrets.ENDPOINT }}
          tenant_id: ${{ secrets.TENANT_ID }}
          quiet: "true"
          soft_fail: "true"
```

Configuring IaC scan in Jenkins

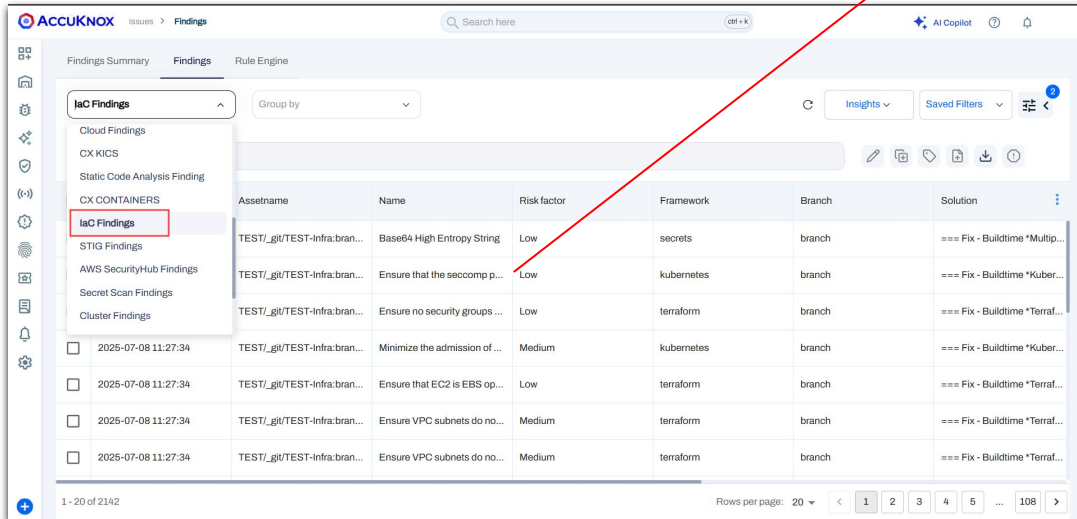
- Jenkins Dashboard → *Manage Jenkins* > *Plugins* > *Advanced* → Upload & deploy
- Open your Jenkins job → *Add build step* → Select AccuKnox IaC scan → Fill required details → Trigger the job



[Visit Help Docs for complete guidance](#)

View Findings in Findings page

- Go to the AccuKnox > Findings and select the **IAC Findings** from the drop down menu.
- To view detailed information about a finding, click on the finding and then select the arrow icon.



AccuKnox Findings

Findings Summary Findings Rule Engine

Search here

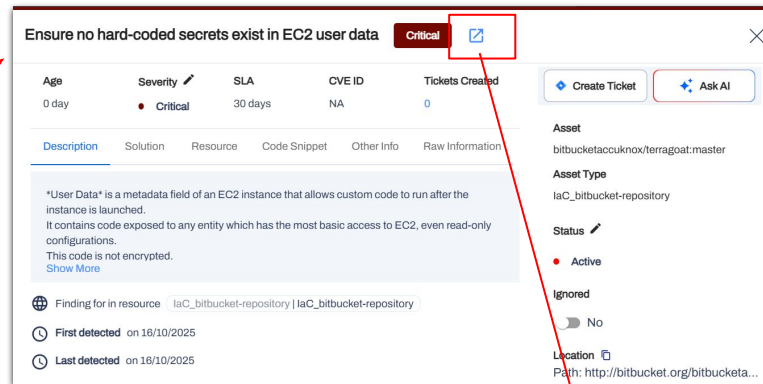
Group by

IaC Findings

Assetname	Name	Risk factor	Framework	Branch	Solution
TEST_gli/TEST-infra:bran...	Base64 High Entropy String	Low	secrets	branch	=== Fix - Buildtime *Multip...
TEST_gli/TEST-infra:bran...	Ensure that the seccomp p...	Low	kubernetes	branch	=== Fix - Buildtime *Kuber...
TEST_gli/TEST-infra:bran...	Ensure no security groups ...	Low	terraform	branch	=== Fix - Buildtime *Terra...
2025-07-08 11:27:34	TEST_gli/TEST-infra:bran...	Minimize the admission of ...	kubernetes	branch	=== Fix - Buildtime *Kuber...
2025-07-08 11:27:34	TEST_gli/TEST-infra:bran...	Ensure that EC2 is EBS op...	terraform	branch	=== Fix - Buildtime *Terra...
2025-07-08 11:27:34	TEST_gli/TEST-infra:bran...	Ensure VPC subnets do no...	terraform	branch	=== Fix - Buildtime *Terra...
2025-07-08 11:27:34	TEST_gli/TEST-infra:bran...	Ensure VPC subnets do no...	terraform	branch	=== Fix - Buildtime *Terra...

1 - 20 of 2142

Rows per page: 20



Ensure no hard-coded secrets exist in EC2 user data Critical

Age: 0 day Severity: Critical SLA: 30 days CVE ID: NA Tickets Created: 0

Create Ticket Ask AI

Description Solution Resource Code Snippet Other Info Raw Information

"User Data" is a metadata field of an EC2 instance that allows custom code to run after the instance is launched. It contains code exposed to any entity which has the most basic access to EC2, even read-only configurations. This code is not encrypted. [Show More](#)

Finding for in resource: laC_bitbucket-repository | laC_bitbucket-repository

First detected on 18/10/2025

Last detected on 18/10/2025

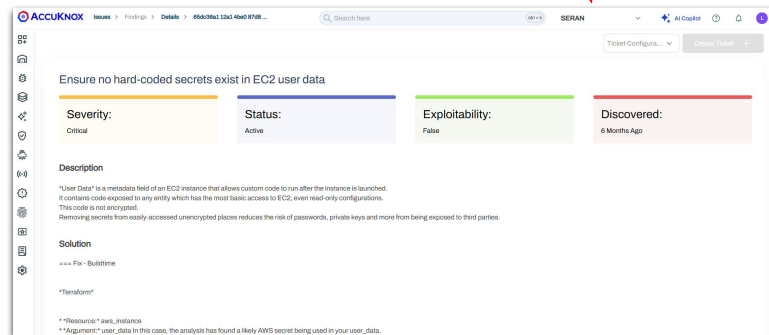
Asset: bitbucketaccuknox/terragoat:master

Asset Type: laC_bitbucket-repository

Status: Active

Ignored: No

Location: Path: http://bitbucket.org/bitbucketa...



AccuKnox Findings Details

Search here

Severity: Critical Status: Active Exploitability: False Discovered: 6 Months Ago

Description

"User Data" is a metadata field of an EC2 instance that allows custom code to run after the instance is launched. It contains code exposed to any entity which has the most basic access to EC2, even read-only configurations. This code is not encrypted. Removing secrets from easily-accessed unencrypted places reduces the risk of passwords, private keys and more from being exposed to third parties.

Solution

=== Fix - Buildtime

Resource aws_instance

**Argument* user_data In this case, the analysis has found a likely AWS secret being used in your user_data

Use Case 1

Public Cloud Exposure

Use Case 2

Privilege Escalation Enabled

Use Case 3

Credential Exposure

Use case 1: Public Cloud Exposure

- S3 bucket is misconfigured without restrict_public_buckets enabled, allowing potential public access.
- This can expose sensitive data to the internet, leading to data breaches and security risks.
- AccuKnox IaC scan detects this misconfiguration early in the pipeline and helps fix it before deployment

Ensure S3 bucket has 'restrict_public_buckets' enabled

Medium🔗✕

Age
164 days

Severity
● Medium

SLA
60 days

CVE ID
NA

Tickets Created
0

🔗 Create Ticket🔗 Ask AI

Description

Solution

Resource

Code Snippet

Other Info

Raw Information

The S3 Block Public Access configuration enables specifying whether S3 should restrict public bucket policies for buckets in this account. Setting RestrictPublicBucket to TRUE restricts access to buckets with public policies to only AWS services and authorized users within this account. Enabling this setting does not affect previously stored bucket policies. Public and cross-account access within any public bucket policy, including non-public delegation to specific accounts, is blocked.

Show Less

🌐 Finding for in resource

laC_github-repository | laC_github-repository

🕒 First detected on 19/05/2025

🕒 Last detected on 30/10/2025

Compliance Frameworks

No compliance found

Asset Information

▶ 📄

Asset

r3d-shadow/azure-plugin-test:HEAD

Asset Type

laC_github-repository

Status

● Active

Ignored

🔌 No

Location

Path: https://github.com/r3d-shadow/azu...

Notes

🕒

Add Comments and Press Ctrl + Enter to Submit

Description

Solution

Resource

Code Snippet

Other Info

Raw Information

=== Fix - Buildtime

Terraform

```
**Resource:* aws_s3_bucket_public_access_block
**Arguments:* restrict_public_buckets

[source,go]
----
resource "aws_s3_bucket_public_access_block" "artifacts" {
  ...
+ restrict_public_buckets = true
}
----
```

CloudFormation

```
**Resource:* AWS::S3::Bucket
**Arguments:* Properties.PublicAccessBlockConfiguration.RestrictPublicBuckets

[source,yaml]
----
Type: 'AWS::S3::Bucket'
```


Use case 2: Privilege Escalation Enabled

- Containers are configured with allowPrivilegeEscalation enabled, allowing processes to gain higher privileges than intended
- This can lead to privilege escalation attacks, container breakout, and compromise of the underlying host system
- AccuKnox IaC scan detects this insecure setting and enforces disabling privilege escalation to maintain least privilege security

Containers should not run with allowPrivilegeEscalation

Medium

Age	Severity	SLA	CVE ID	Tickets Created
618 days	Medium	60 days	NA	1

DescriptionSolutionResourceCode SnippetOther InfoRaw Information

The "AllowPrivilegeEscalation" Pod Security Policy controls whether or not a user is allowed to set the security context of a container to "True". Setting it to "False" ensures that no child process of a container can gain more privileges than its parent. We recommend you to set "AllowPrivilegeEscalation" to "False", to ensure "RunAsUser" commands cannot bypass their existing sets of permissions.[Show Less](#)

Finding for in resource: iaC_github-repository | iaC_github-repository

First detected on 12/08/2024

Last detected on 22/04/2026

Compliance Frameworks
No compliance found

Asset Information
iaC_github-repository

Create TicketAsk AI

Asset
nyrahul/nephio.git:main

Asset Type
iaC_github-repository

Status
Active

Ignored
No

Location
Path: https://github.com/nyrahul/nephio.git/b...

Notes
Add Comments and Press Ctrl + Enter to Submit

Containers should not run with allowPrivilegeEscalation

Medium

Age	Severity	SLA	CVE ID	Tickets Created
618 days	Medium	60 days	NA	1

DescriptionSolutionResourceCode SnippetOther InfoRaw Information

=== Fix - Buildtime

Kubernetes

**Resource:* Container

**Arguments:* allowPrivilegeEscalation (Optional) If false, the pod can not request to allow privilege escalation. Default to true.

[source.yaml]

apiVersion: v1

kind: Pod

metadata:

name:

spec:

containers:

- name:

image:

securityContext:

+ allowPrivilegeEscalation: false

[Show Less](#)

Use case 3: Credential Exposure

- IAM policies are misconfigured in a way that may allow exposure of sensitive credentials
- This can lead to unauthorized access, data breaches, and potential compromise of cloud resources
- AccuKnox IaC scan detects such policy misconfigurations early and helps prevent credential exposure

Ensure IAM policies does not allow credentials exposure High

Age
0 day

Severity
High

SLA
45 days

CVE ID
NA

Tickets Created
0

Create Ticket

Ask AI

Description

Solution

Resource

Code Snippet

Other Info

Raw Information

This policy is used to verify if Identity and Access Management (IAM) policies are configured in a way that prevents the exposure of credentials. This is paramount for security as exposure of credentials could allow unauthorized users access to sensitive resources and operations. This includes viewing, modifying or deleting data, which can expose the organization to a range of risks, from data breaches to the potential shut down of systems. Therefore, it's crucial to ensure IAM policies are correctly configured to prevent credentials exposure.

[Show Less](#)

Finding for in resource

laC_bitbucket-repository | laC_bitbucket-repository

First detected

on 17/10/2025

Last detected

on 17/10/2025

Compliance Frameworks

No compliance found

Asset Information

Asset

vickyaccuknox/terragoat:master

Asset Type

laC_bitbucket-repository

Status

Active

Ignored

No

Location

Path: http://bitbucket.org/vickyaccuknox/terr...

Notes

Add Comments and Press Ctrl + Enter to Submit

Description

Solution

Resource

Code Snippet

Other Info

Raw Information

=== Fix - Buildtime

Terraform

* *Resource:* aws_iam_policy

* *Arguments:* policy

To fix this issue, you need to review and ensure that the IAM policies do not allow the exposure of credentials. IAM Policies should enforce the least privileges principle.

[source,hol]

resource "aws_iam_policy" "example" {

name = "example"

path = "/"

description = "An example policy"

policy = <

[Show Less](#)



SAST Scan

ASPM Integration

Step 1: Add [AccuKnox SAST scan GitHub action](#) to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically `.github/workflows/your-workflow.yml`).
- Configure these Parameters as GitHub Secrets
``ACCUKNOX_LABEL``, ``ACCUKNOX_ENDPOINT``, ``ACCUKNOX_TOKEN``

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select SAST Findings.
- Click on any finding that interests you to view detailed information and recommendations.

```
name: AccuKnox Opengrep SAST

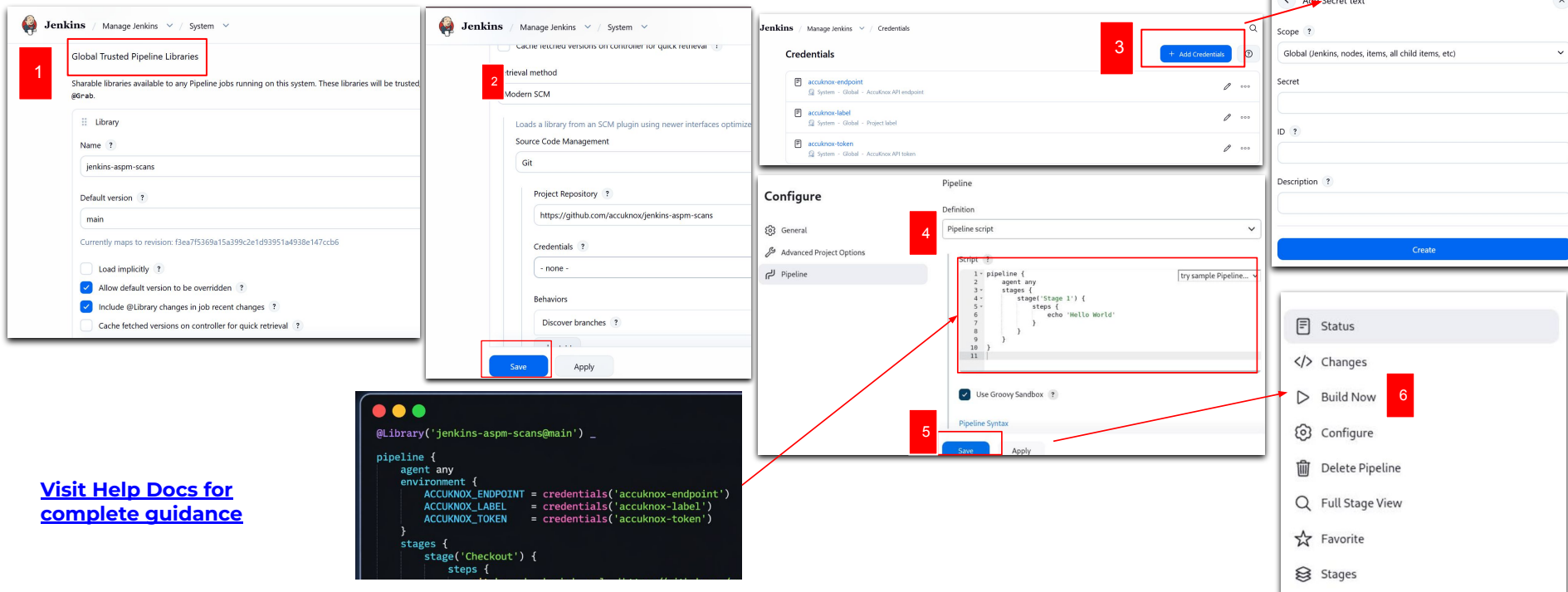
on:
  push:
    branches:
      - opengrep
  pull_request:
    branches:
      - opengrep

jobs:
  accuknox-cicd:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: "Run AccuKnox SAST: Opengrep"
        uses: accuknox/sast-scan-opengrep-action@v1.0.1
        with:
          accuknox_endpoint: "${{ secrets.ACCUKNOX_ENDPOINT }}"
          accuknox_token: "${{ secrets.ACCUKNOX_TOKEN }}"
          accuknox_label: "${{ secrets.ACCUKNOX_LABEL }}"
          soft_fail: true
```

Configuring SAST OpenGrep scan in Jenkins

- Manage Jenkins → System → Global Trusted Pipeline Libraries
- Manage Jenkins → Credentials → Global → Add Credentials
- Go to Configure → pipeline → In the script box that appears, paste your pipeline script.
- After adding the script, click "Save" or "Apply."
- Go to the newly created pipeline and click "Build Now" to run the script.



1 Global Trusted Pipeline Libraries

2 Source Code Management

3 Add Credentials

4 Pipeline script

5 Save

6 Build Now

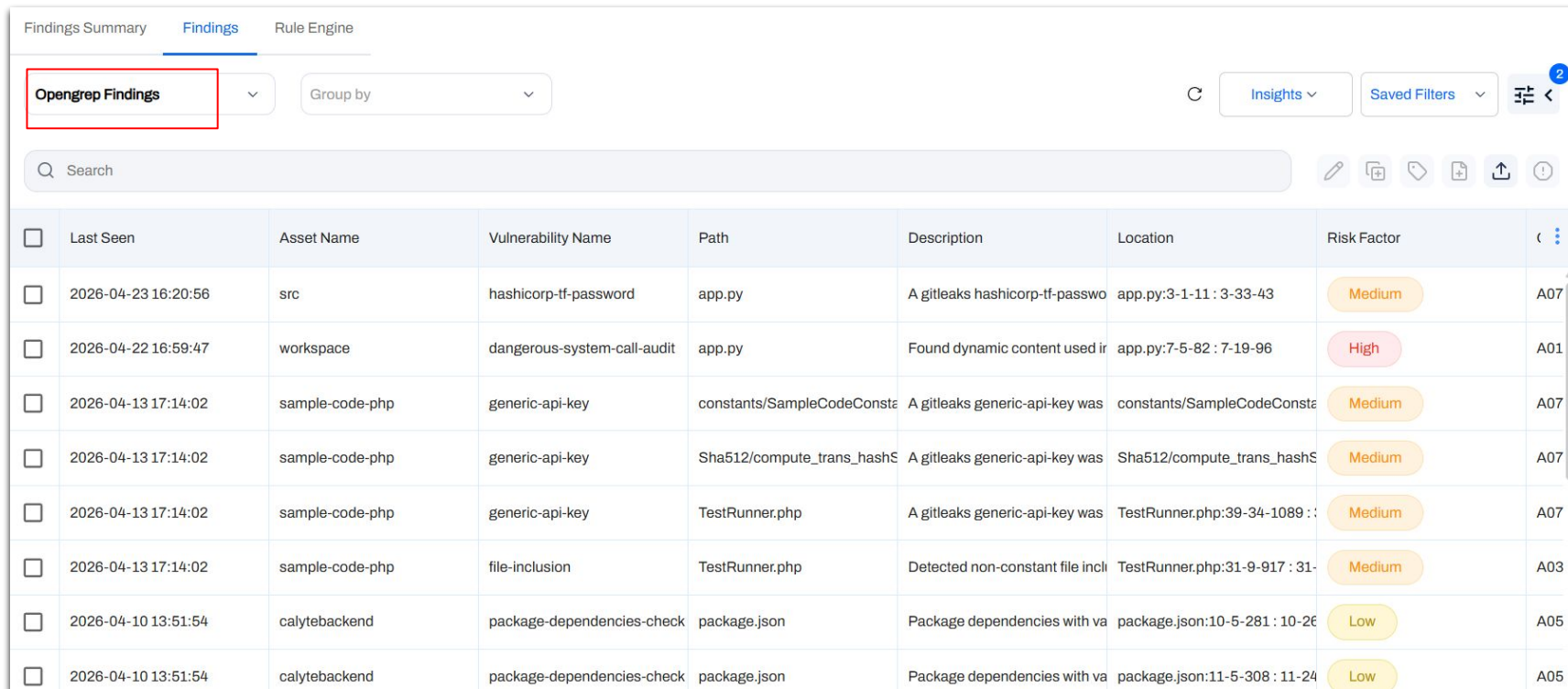
```
@Library('jenkins-aspm-scans@main') _
pipeline {
  agent any
  environment {
    ACCUKNOX_ENDPOINT = credentials('accuknox-endpoint')
    ACCUKNOX_LABEL = credentials('accuknox-label')
    ACCUKNOX_TOKEN = credentials('accuknox-token')
  }
  stages {
    stage('Checkout') {
      steps {

```

[Visit Help Docs for complete guidance](#)

View Findings in Findings page

- Go to the AccuKnox > Findings and select the **OpenGrep findings** here.



The screenshot shows the AccuKnox Findings page. At the top, there are tabs for 'Findings Summary', 'Findings' (which is active), and 'Rule Engine'. Below the tabs, there is a filter dropdown menu set to 'OpenGrep Findings', a 'Group by' dropdown, and buttons for 'Insights', 'Saved Filters', and a filter icon. A search bar is located below the filters. The main content is a table of findings with columns: Last Seen, Asset Name, Vulnerability Name, Path, Description, Location, Risk Factor, and an ID column. The table contains 8 rows of findings, each with a checkbox in the first column. The risk factors are categorized as Medium, High, or Low.

☐	Last Seen	Asset Name	Vulnerability Name	Path	Description	Location	Risk Factor	ID
☐	2026-04-23 16:20:56	src	hashicorp-tf-password	app.py	A gitleaks hashicorp-tf-passwo	app.py:3-1-11 : 3-33-43	Medium	A07
☐	2026-04-22 16:59:47	workspace	dangerous-system-call-audit	app.py	Found dynamic content used in	app.py:7-5-82 : 7-19-96	High	A01
☐	2026-04-13 17:14:02	sample-code-php	generic-api-key	constants/SampleCodeConste	A gitleaks generic-api-key was	constants/SampleCodeConste	Medium	A07
☐	2026-04-13 17:14:02	sample-code-php	generic-api-key	Sha512/compute_trans_hashS	A gitleaks generic-api-key was	Sha512/compute_trans_hashS	Medium	A07
☐	2026-04-13 17:14:02	sample-code-php	generic-api-key	TestRunner.php	A gitleaks generic-api-key was	TestRunner.php:39-34-1089 :	Medium	A07
☐	2026-04-13 17:14:02	sample-code-php	file-inclusion	TestRunner.php	Detected non-constant file incl	TestRunner.php:31-9-917 : 31-	Medium	A03
☐	2026-04-10 13:51:54	calytebackend	package-dependencies-check	package.json	Package dependencies with va	package.json:10-5-281 : 10-26	Low	A05
☐	2026-04-10 13:51:54	calytebackend	package-dependencies-check	package.json	Package dependencies with va	package.json:11-5-308 : 11-24	Low	A05

Use Case 1

Privilege Escalation

Use Case 2

XML External Entity(XXE) Injection

Use Case 3

Hard Coded Password

Use case 1: Privilege Escalation

- An IAM policy in this code is vulnerable to privilege escalation attack.
- AccuKnox identifies this vulnerability and suggests a solution.

This policy is vulnerable to the "EC2" privilege escalation vector. Remove permissions or restrict the set of resources they apply to.

Critical  

Description	Result	Solution	References	Source Code
<p>Within IAM, identity-based policies grant permissions to users, groups, or roles, and enable specific actions to be performed on designated resources. When an identity policy inadvertently grants more privileges than intended, certain us Show More...				

Details [+ Create Ticket](#)

Asset
gitlab-sast-testing

Asset Type
static_code_Software

Status 

Use case 2: XML External Entity (XXE) injection

- This code have an XML parsing vulnerability XML External Entity injection.
- AccuKnox identifies the vulnerability and proposes a solution.

Disable access to external entities in XML parsing.

critical

Description

Result

Solution

References

Source Code

<p>This vulnerability allows the usage of external entities in XML.</p>
<h2>Why is this an issue?</h2>
<p>External Entity Processing allows for XML parsing with the involvement of external entities. However, when this functionality is enabl
[Show More...](#)

Details

+ Create Ticket

Asset

test-vulnerable-code-snippets

Asset Type

static_code_Software

Status 

Use case 3: Hard Coded Password

- There is a hardcoded password in the source code.
- AccuKnox identifies the vulnerability, and proposes a solution.

Review this potentially hardcoded credential. Follow 🔗 ×

Age	Severity	SLA	CVE ID	Tickets Created
0 day	● Critical	30 days	CWE-798	0

DescriptionSolutionClassificationSource CodeRaw Information

```
1  );
2  new mysql.Database(
3  {
4    hostname: 'localhost',
5    user: 'user',
6    password: 'password',
7    database: 'test'
8  }
9  ).connect(function(error)
10 {
11   var the_Query =
12   "INSERT INTO Customers (CustomerName, ContactName) VALUES ('Tom'," +
13   valTom + ")";
14   this.query(the_Query).execute(function(error, result)
15   {
16
```

Create TicketAsk AI

Asset
test-vulnerable-code-snippets

Asset Type
static_code_Software

Status
Active

Ignored
☐ No

Location
test:test-vulnerable-code-snippets:SQL...

Notes
Add Comments and Press Ctrl + Enter to Submit



DAST Scan

ASPM Integration

Configuring the DAST scan in GitHub actions



Step 1: Add [AccuKnox DAST scan GitHub action](#) to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically .github/workflows/your-workflow.yml)
- After the build step, add the AccuKnox container scan GitHub Action.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox:

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select DAST Findings.
- Click on any finding that interests you to view detailed information and recommendations.

[Visit Help Docs for complete guidance](#)

```
jobs:
  zap:
    runs-on: ubuntu-latest
    steps:
      - name: set permissions
        run: sudo chmod -R 777 .

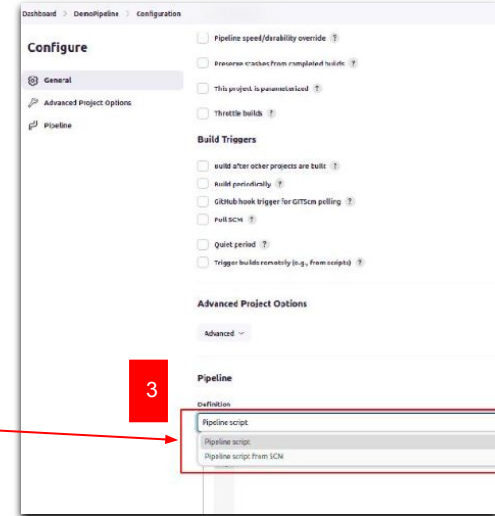
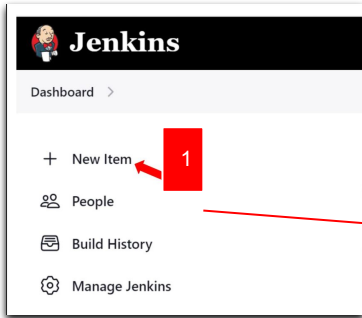
      - name: workspace permissions
        run: sudo chmod -R 777 ${github.workspace}

      - name: zap scan
        run: |
          docker run --rm -v ${github.workspace}:/zap/wrk:rw -t zapproxy/zap-stable zap-
          baseline.py \
            -t <url that you want to scan> \
            -J report.json
            -I

      - name: zap
        run: |
          cd ${GITHUB_WORKSPACE}
          curl --location --request POST 'https://${secrets.accuknox_url}
          }}/api/v1/artifact?tenant_id=${secrets.tenant_id}
          }}&data_type=ZAP&label_id=dasttest&save_to_s3=false' \
            --header 'Tenant-Id: ${secrets.tenant_id}' \
            --header 'Authorization: Bearer ${secrets.token}' \
            --form 'file=@\"report.json\"'
```


Configuring the DAST scan in Jenkins

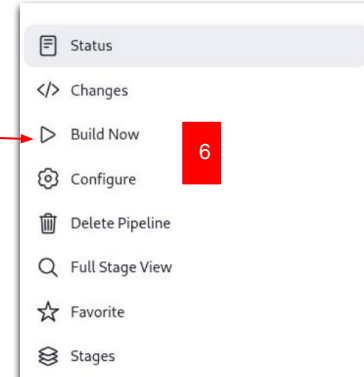
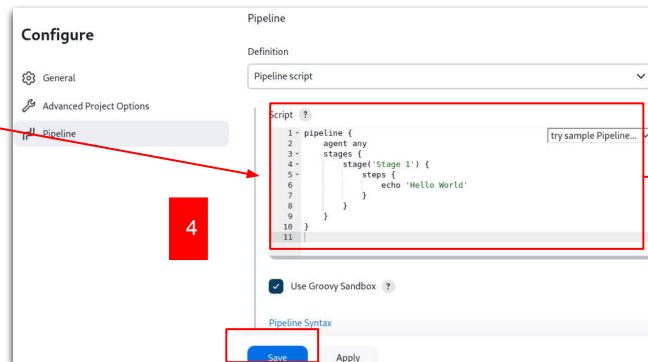
- Go to Jenkins Dashboard → Click *New Item* → Enter name → Select *Pipeline* → Click OK
- In Pipeline configuration → Select *Pipeline script* → Paste your script → Save/Apply
- Open the pipeline → Click *Build Now* to execute



```
pipeline {
  agent any

  environment {
    TOKEN = credentials('ACCUKNOX_TOKEN')
    END_POINT = 'cspm.demo.accuknox.com'
    TENANT_ID = '167'
    LABEL = 'SPOC'
  }

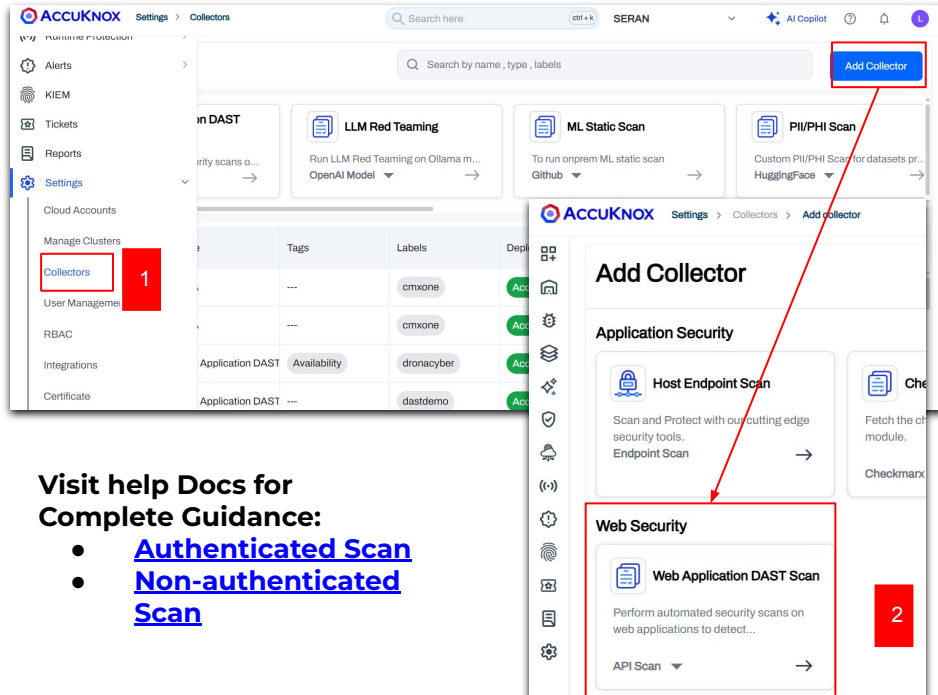
  stages {
    stage('Checkout Repository') {
      steps {
        git 'https://github.com/safeer-accuknox/mfa-dast-'
      }
    }
  }
}
```



[Visit Help Docs for complete guidance](#)

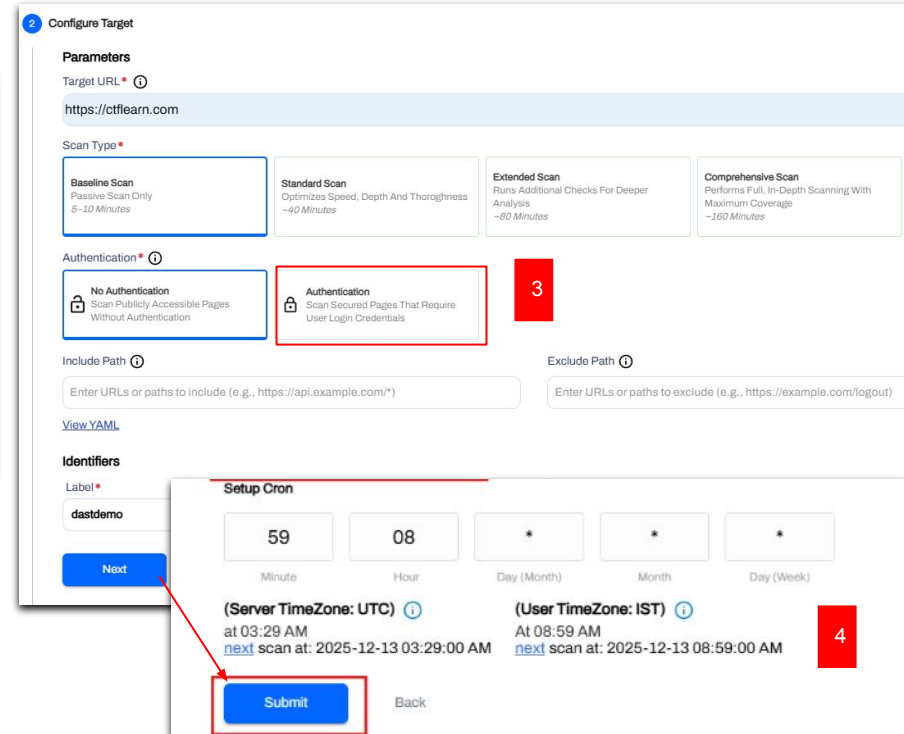
Authenticated/Non Authenticated DAST Collector Scan

- Go to AccuKnox → Settings > Collectors > Add Collector → Select **Web Application DAST Scan** and give a name
- Select **Authenticated** (enter Login URL & credentials) or **Non-Authenticated** scan
- Fill required scan settings and target details.
- Set cron job (if needed) and click **Submit**



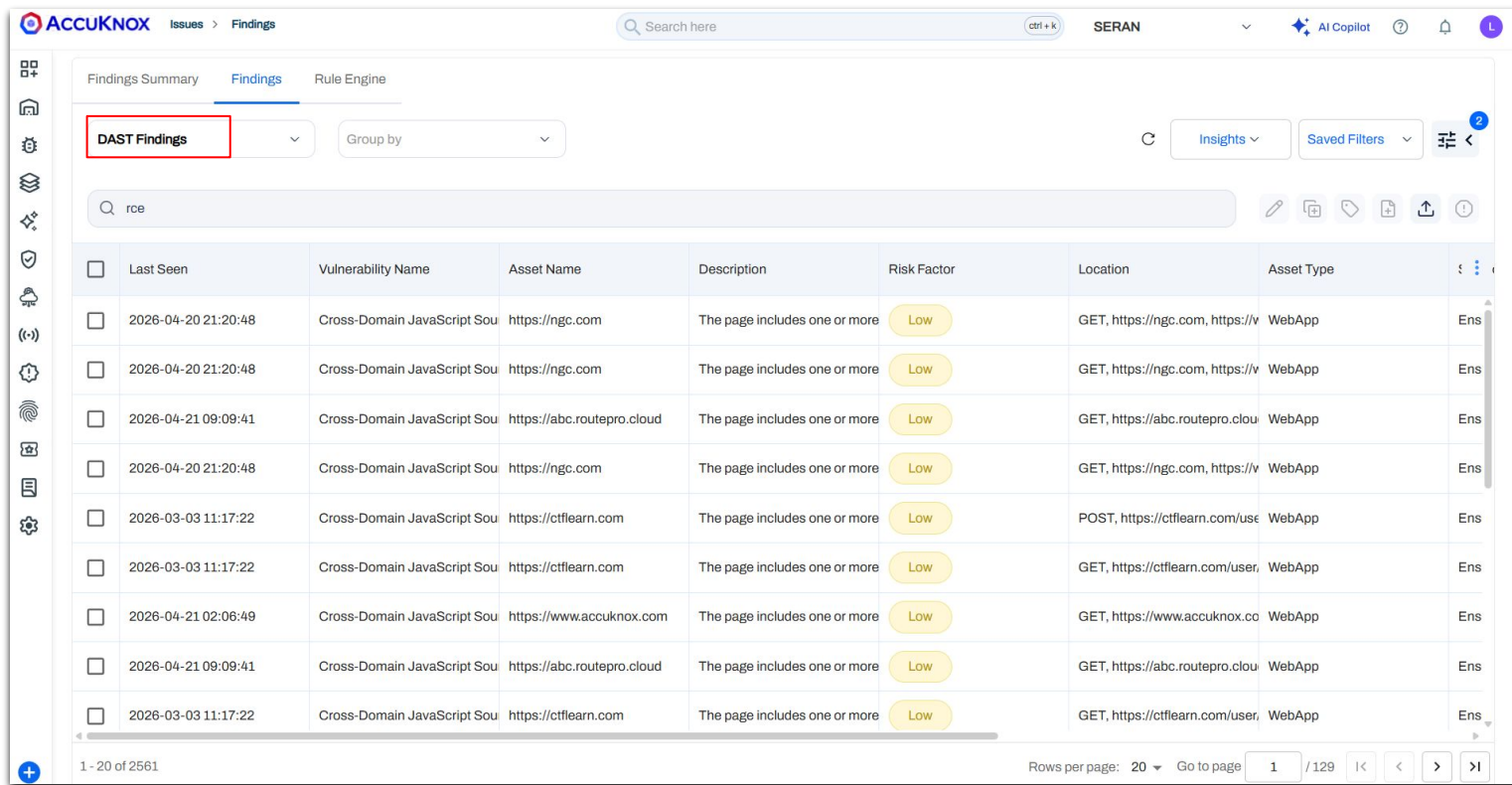
Visit help Docs for Complete Guidance:

- [Authenticated Scan](#)
- [Non-authenticated Scan](#)



View Findings in Findings page

- To get the findings, go to the AccuKnox > Findings and select the **DAST findings here**.



The screenshot displays the AccuKnox Findings page. The 'Findings' tab is active, and the 'DAST Findings' filter is selected. The table below lists various findings, all categorized as 'Low' risk factors. The findings are related to Cross-Domain JavaScript Source (Sou) vulnerabilities across different assets.

	Last Seen	Vulnerability Name	Asset Name	Description	Risk Factor	Location	Asset Type	
☐	2026-04-20 21:20:48	Cross-Domain JavaScript Sou	https://ngc.com	The page includes one or more	Low	GET, https://ngc.com, https://v	WebApp	Ens
☐	2026-04-20 21:20:48	Cross-Domain JavaScript Sou	https://ngc.com	The page includes one or more	Low	GET, https://ngc.com, https://v	WebApp	Ens
☐	2026-04-21 09:09:41	Cross-Domain JavaScript Sou	https://abc.routeopro.cloud	The page includes one or more	Low	GET, https://abc.routeopro.clou	WebApp	Ens
☐	2026-04-20 21:20:48	Cross-Domain JavaScript Sou	https://ngc.com	The page includes one or more	Low	GET, https://ngc.com, https://v	WebApp	Ens
☐	2026-03-03 11:17:22	Cross-Domain JavaScript Sou	https://ctflearn.com	The page includes one or more	Low	POST, https://ctflearn.com/use	WebApp	Ens
☐	2026-03-03 11:17:22	Cross-Domain JavaScript Sou	https://ctflearn.com	The page includes one or more	Low	GET, https://ctflearn.com/user,	WebApp	Ens
☐	2026-04-21 02:06:49	Cross-Domain JavaScript Sou	https://www.accuknox.com	The page includes one or more	Low	GET, https://www.accuknox.co	WebApp	Ens
☐	2026-04-21 09:09:41	Cross-Domain JavaScript Sou	https://abc.routeopro.cloud	The page includes one or more	Low	GET, https://abc.routeopro.clou	WebApp	Ens
☐	2026-03-03 11:17:22	Cross-Domain JavaScript Sou	https://ctflearn.com	The page includes one or more	Low	GET, https://ctflearn.com/user,	WebApp	Ens

1 - 20 of 2561

Rows per page: 20 Go to page 1 / 129

Use Case 1**Cross Origin Resource Sharing (CORS) Misconfiguration****Use Case 2****File Inclusion Vulnerability****Use Case 3****Cross site scripting vulnerability**

Use case 1: Cross Origin Resource Sharing (CORS) misconfiguration

- This web application is vulnerable to CORS.
- CORS can lead to so many issues such as unauthorized access, cross site scripting, session hijacking and many other issues.
- AccuKnox identifies this vulnerability and proposes the solution.

Cross-Domain Misconfiguration

Critical

Age	Severity	SLA	CVE ID	Tickets Created
0 day	Critical	30 days	CWE-264	0

Description

Solution

Classification

Evidence

Request

Response

Raw Information

Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Finding for in resource WebApp | WebApp

First detected on 01/12/2025

Last detected on 01/12/2025

Compliance Frameworks

No compliance found

Asset Information

Create Ticket

Ask AI

Asset

https://juice-shop.herokuapp.com

Asset Type

WebApp

Location

GET, https://juice-shop.herokuapp.com/runtime.js,

Status

Active

Ignored

No

Location

GET, https://juice-shop.herokuapp.com/r...

Notes

Ask AI

Remediation Steps for Cross-Domain Misconfiguration on <https://juice-shop.herokuapp.com>

Affected Asset:

- URL: <https://juice-shop.herokuapp.com>
- Vulnerable Element: HTTP Header - Access-Control-Allow-Origin: *

Issue Summary: The identified vulnerability is due to a permissive Cross-Origin Resource Sharing (CORS) policy on the Juice Shop application hosted at the specified URL. The current configuration allows any domain to access resources, which could potentially expose sensitive data to unauthorized domains.

Remediation Steps:

- Understand Your CORS Requirements:
 - Identify which domains need to legitimately access your resources. This often includes your own domains, trusted third-party services, or specific partners.
- Restrict Access-Control-Allow-Origin:
 - Update the CORS policy to specify a more restrictive list of allowed domains instead of using the wildcard *. For example, if only `https://trusted-partner.com` should access resources, configure:

plaintext Copy code

Access-Control-Allow-Origin: https://trusted-partner.com

Use case 2: File inclusion vulnerability

- This web application is vulnerable to file inclusion.
- In file inclusion attack, and attacker can access files stored on a web server.
- This can lead to sensitive data leakage and source code leakage.
- AccuKnox identifies this vulnerability, and proposes a solution.

Cross-Domain JavaScript Source File Inclusion

Low

Age	Severity	SLA	CVE ID	Tickets Created
125 days	Low	90 days	CWE-829	1

DescriptionSolutionClassificationEvidenceRequestResponseRaw Information

The page includes one or more script files from a third-party domain.

Finding for in resource WebApp | WebApp

First detected on 16/12/2025

Last detected on 20/04/2026

Compliance Frameworks
No compliance found

Asset Information
🔗

Create Ticket

Ask AI

Asset

https://ngc.com

Asset Type

WebApp

Location

GET, https://ngc.com, https://www.northropgrumma.com/_next/static/chunks/framework-e952fed463eb8e34.js

Status

Active

Ignored

No

Location

GET, https://ngc.com, https://www.northropg...

Cross-Domain JavaScript Source File Inclusion

Low

Age	Severity	SLA	CVE ID	Tickets Created
125 days	Low	90 days	CWE-829	1

DescriptionSolutionClassificationEvidenceRequest

Ensure JavaScript source files are loaded from only trusted sources, and the sources can't be controlled by end users of the application.

Use case 3: Cross site scripting vulnerability

- This web application have a cross site scripting vulnerability (XSS)
- XSS allows an attacker to inject arbitrary javascript code into a webpage
- AccuKnox identifies the vulnerability and proposes a solution to it

User Controllable HTML Element Attribute (Potential XSS)

Informational

Age	Severity	SLA	CVE ID	Tickets Created
0 day	Informational	120 days	CWE-20	0

Description

Solution

Classification

Evidence

Request

Response

Raw Information

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Finding for in resource WebApp | WebApp

First detected on 20/08/2025

Last detected on 20/08/2025

Compliance Frameworks
No compliance found

Create Ticket

Ask AI

Asset
http://testasp.vulnweb.com

Asset Type
WebApp

Location
http://testasp.vulnweb.com/showthread.asp?id=10

Status
Active

Ignored
☐ No

Ask AI

Remediation Guide for User Controllable HTML Element Attribute (Potential XSS)

Affected Asset:
URL: <http://testasp.vulnweb.com>

Vulnerable HTML Tag:
The specific vulnerable HTML tag is not directly provided in the finding. Therefore, a thorough review of the code handling user input, especially within HTML attributes, is necessary.

Remediation Steps:

- Input Validation:**
 - Implement strict input validation on all user-supplied data, especially those coming from query string parameters and POST data.
 - Use a whitelist approach where only expected and safe input formats are allowed.
 - Reference: [OWASP Input Validation Cheat Sheet](#)
- Output Encoding:**
 - Ensure that data is properly encoded before being included in HTML attributes.
 - Use a library or framework that provides context-sensitive encoding functions to prevent XSS attacks. For instance, JavaScript or HTML attribute encoding.
 - Reference: [OWASP XSS Prevention Cheat Sheet](#)



Secret Scanning

ASPM Integration

Configuring the Secret scan in GitHub actions

Step 1: Add [AccuKnox Secret Scan GitHub action](#) to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically .github/workflows/your-workflow.yml).
- After the build step, add the AccuKnox container scan GitHub Action.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

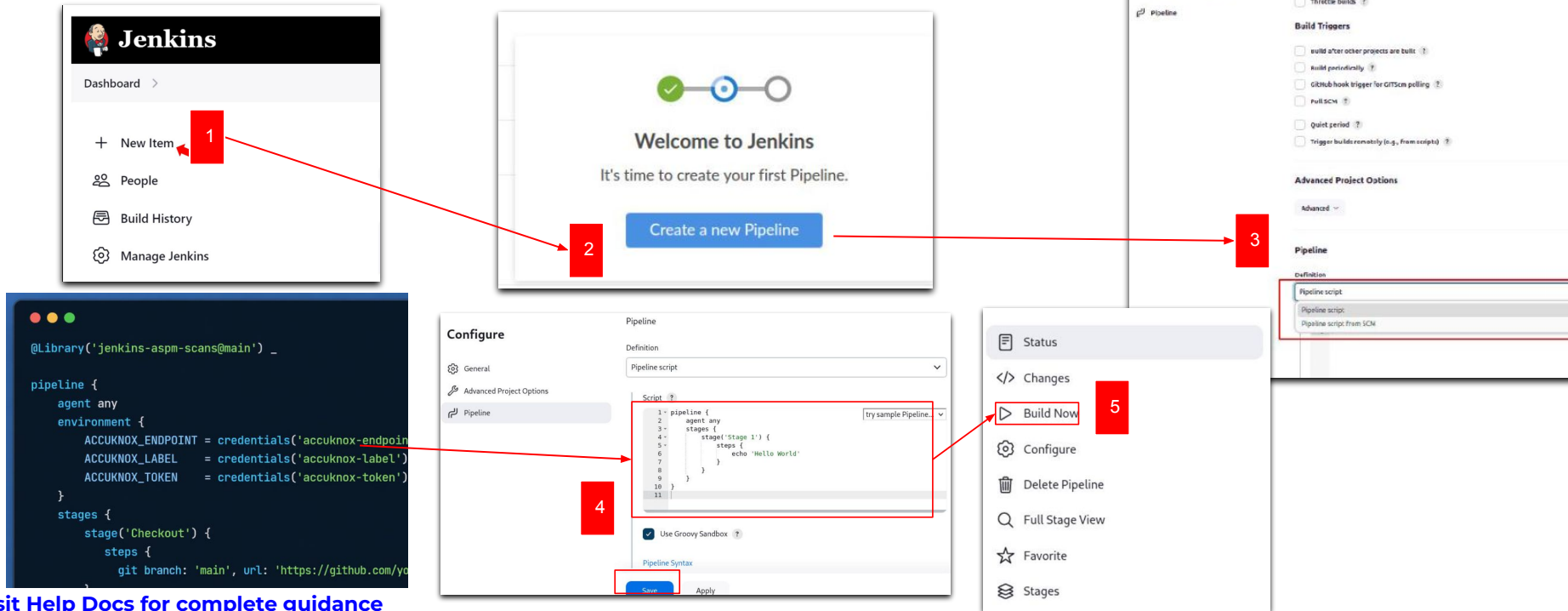
- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select Secret scan Findings.
- Click on any finding that interests you to view detailed information and recommendations.

[Visit Help Docs for complete guidance](#)

```
name: AccuKnox Secret Scan Workflow
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Run Secret Scan
        uses: accuknox/secret-scan-action@latest
        with:
          branch: "main"
          results: ""
          exclude_paths: "tests/,docs/"
```

Configuring the Secret Scan in Jenkins

- Go to Jenkins Dashboard → Click *New Item* → Enter name → Select *Pipeline* → Click OK
- In Pipeline configuration → Select *Pipeline script* → Paste your script → Save/Apply
- Open the pipeline → Click *Build Now* to execute



Step 1: Jenkins Dashboard → Click *New Item*

Step 2: Welcome to Jenkins → It's time to create your first Pipeline. → Create a new Pipeline

Step 3: Pipeline configuration → Select *Pipeline script*

Step 4: Pipeline configuration → Paste your script

```
@Library('jenkins-aspn-scans@main') _

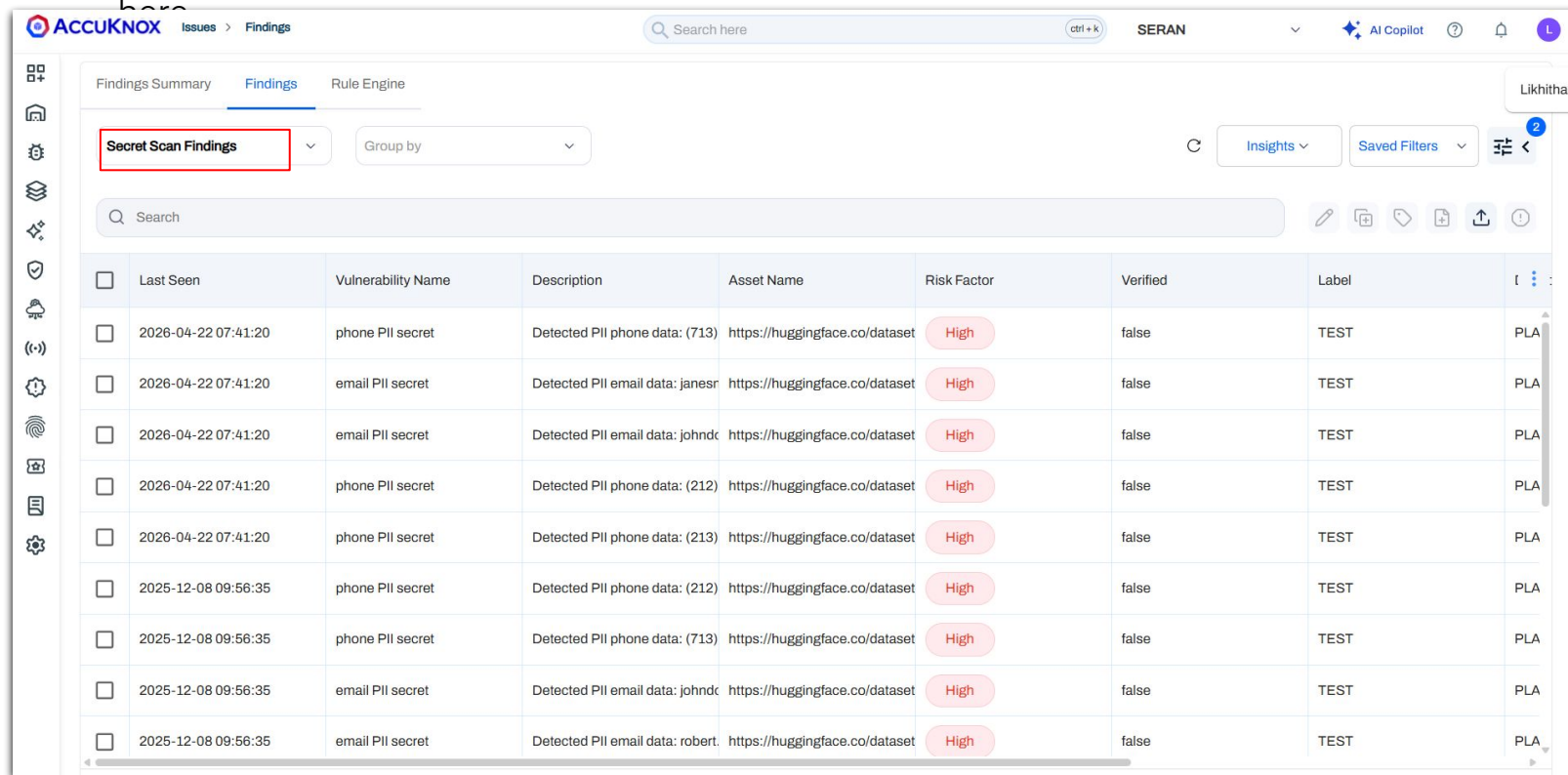
pipeline {
  agent any
  environment {
    ACCUKNOX_ENDPOINT = credentials('accuknox-endpoint')
    ACCUKNOX_LABEL     = credentials('accuknox-label')
    ACCUKNOX_TOKEN     = credentials('accuknox-token')
  }
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/you/repo'
      }
    }
  }
}
```

Step 5: Pipeline configuration → Click *Build Now*

[Visit Help Docs for complete guidance](#)

View Findings in Findings page

- To get the findings, go to the AccuKnox > Findings and select **Secret Scan Findings**



The screenshot displays the AccuKnox Findings page. The 'Secret Scan Findings' filter is selected and highlighted with a red box. The table below lists various findings, all with a 'High' risk factor and 'false' verification status.

	Last Seen	Vulnerability Name	Description	Asset Name	Risk Factor	Verified	Label	
☐	2026-04-22 07:41:20	phone PII secret	Detected PII phone data: (713)	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2026-04-22 07:41:20	email PII secret	Detected PII email data: janesr	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2026-04-22 07:41:20	email PII secret	Detected PII email data: johndc	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2026-04-22 07:41:20	phone PII secret	Detected PII phone data: (212)	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2026-04-22 07:41:20	phone PII secret	Detected PII phone data: (213)	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2025-12-08 09:56:35	phone PII secret	Detected PII phone data: (212)	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2025-12-08 09:56:35	phone PII secret	Detected PII phone data: (713)	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2025-12-08 09:56:35	email PII secret	Detected PII email data: johndc	https://huggingface.co/dataset	High	false	TEST	PLA
☐	2025-12-08 09:56:35	email PII secret	Detected PII email data: robert.	https://huggingface.co/dataset	High	false	TEST	PLA

Use Case 1

Sensitive Data Exposure (PII Leakage)

Use Case 2

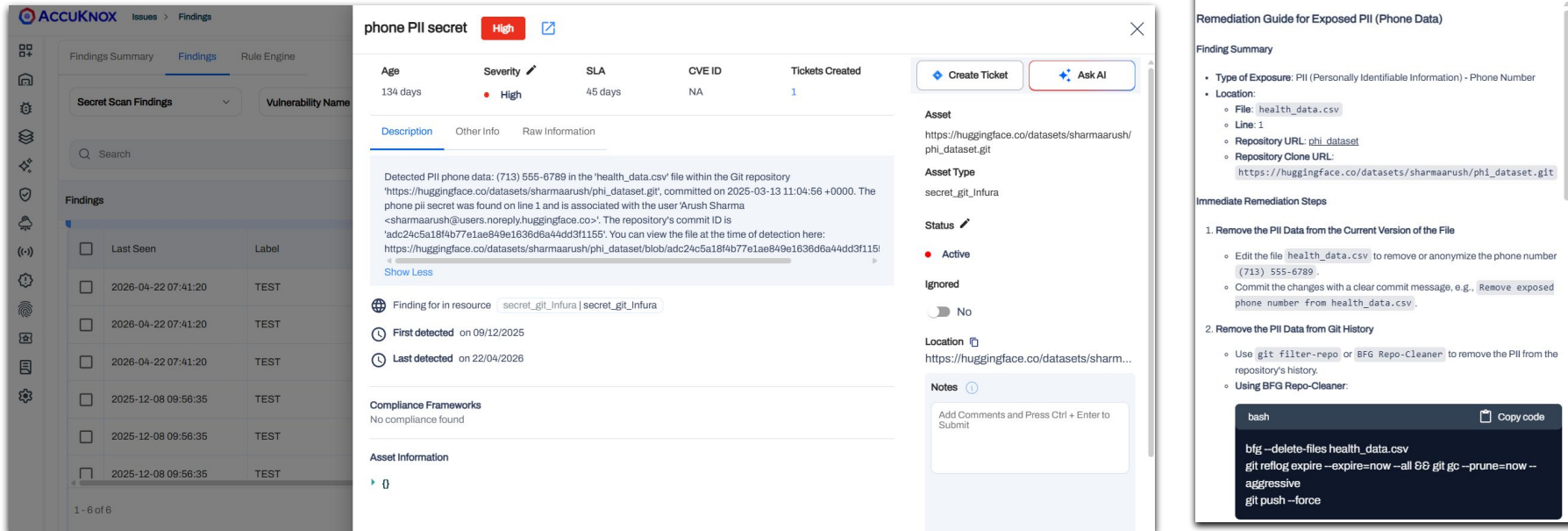
API Keys & Access Tokens Leakage

Use Case 3

Private Key Exposure

Use case 1: Sensitive Data Exposure (PII Leakage)

- AccuKnox identified exposed PII data (phone number) in the repository
- This may lead to privacy violations, data misuse, and compliance risks (GDPR, etc.)
- AccuKnox identifies the vulnerability and proposes a solution to it



The screenshot displays the AccuKnox interface with a sidebar on the left containing navigation icons. The main area is divided into two panels. The left panel, titled 'Findings', shows a table of findings with columns for 'Last Seen', 'Label', and 'Severity'. The right panel, titled 'phone PII secret', shows details for a specific finding. It includes a 'Severity' of 'High', a 'SLA' of '45 days', and a 'CVE ID' of 'NA'. The description states: 'Detected PII phone data: (713) 555-6789 in the 'health_data.csv' file within the Git repository 'https://huggingface.co/datasets/sharmaarush/phi_dataset.git', committed on 2025-03-13 11:04:56 +0000. The phone pii secret was found on line 1 and is associated with the user 'Arush Sharma <sharmaarush@users.noreply.huggingface.co>'. The repository's commit ID is 'adc24c5a18f4b77e1ae849e1636d6a44dd3f1155'. You can view the file at the time of detection here: https://huggingface.co/datasets/sharmaarush/phi_dataset/blob/adc24c5a18f4b77e1ae849e1636d6a44dd3f1155'. Below the description, there are sections for 'Compliance Frameworks' (No compliance found) and 'Asset Information' ({}).

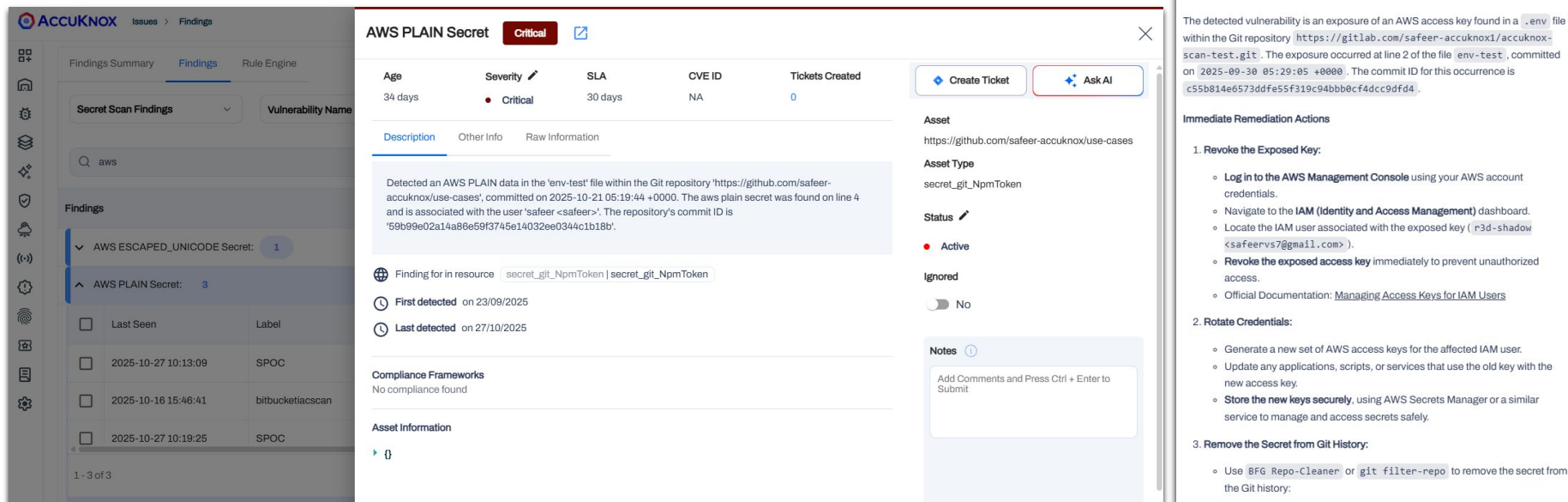
The right panel also includes a 'Remediation Guide for Exposed PII (Phone Data)' section. It provides a 'Finding Summary' with details on the type of exposure (PII - Personally Identifiable Information) and the location (File: health_data.csv, Line: 1). It also lists the repository URL and clone URL. The 'Immediate Remediation Steps' are:

1. Remove the PII Data from the Current Version of the File
 - Edit the file `health_data.csv` to remove or anonymize the phone number (713) 555-6789.
 - Commit the changes with a clear commit message, e.g., `Remove exposed phone number from health_data.csv`.
2. Remove the PII Data from Git History
 - Use `git filter-repo` or `BFG Repo-Cleaner` to remove the PII from the repository's history.
 - Using `BFG Repo-Cleaner`:

```
bash
bfg --delete-files health_data.csv
git reflow expire --expire=now --all && git gc --prune=now --aggressive
git push --force
```

Use case 2: API Keys & Access Tokens Leakage

- AccuKnox detected exposed AWS secret key in the repository
- This may lead to unauthorized cloud access, resource misuse, and financial loss
- AccuKnox proposes the solution to remove the key, revoke and rotate credentials, and use secure secret management.



AccuKnox Issues > Findings

Findings Summary Findings Rule Engine

Secret Scan Findings Vulnerability Name

Q aws

Findings

- ▼ AWS ESCAPED_UNICODE Secret: 1
- ▲ AWS PLAIN Secret: 3

	Last Seen	Label
☐	2025-10-27 10:13:09	SPOC
☐	2025-10-16 15:46:41	bitbucketascan
☐	2025-10-27 10:19:25	SPOC

1 - 3 of 3

AWS PLAIN Secret Critical

Age: 34 days Severity: Critical SLA: 30 days CVE ID: NA Tickets Created: 0

[Description](#) [Other info](#) [Raw Information](#)

Detected an AWS PLAIN data in the 'env-test' file within the Git repository 'https://github.com/safeer-accuknox/use-cases', committed on 2025-10-21 05:19:44 +0000. The aws plain secret was found on line 4 and is associated with the user 'safer <safer>'. The repository's commit ID is '59b99e02a14a89e59f3745e14032ee0344c1b18b'.

Finding for in resource: secret_git_NpmToken | secret_git_NpmToken

First detected on 23/09/2025

Last detected on 27/10/2025

Compliance Frameworks

No compliance found

Asset Information

0

Asset

https://github.com/safeer-accuknox/use-cases

Asset Type

secret_git_NpmToken

Status

Active

Ignored ☐ No

Notes

Add Comments and Press Ctrl + Enter to Submit

Remediation Steps for Exposed AWS Secret

Overview

The detected vulnerability is an exposure of an AWS access key found in a '.env' file within the Git repository <https://gitlab.com/safeer-accuknox1/accuknox-scan-test.git>. The exposure occurred at line 2 of the file 'env-test', committed on 2025-09-30 05:29:05 +0000. The commit ID for this occurrence is c55b814e6573ddf55f319c94bb0cf4dccc9dfd4.

Immediate Remediation Actions

- 1. Revoke the Exposed Key:**
 - Log in to the **AWS Management Console** using your AWS account credentials.
 - Navigate to the **IAM (Identity and Access Management)** dashboard.
 - Locate the IAM user associated with the exposed key (r3d-shadow <safervs7@gmail.com>).
 - Revoke the exposed access key immediately to prevent unauthorized access.
 - Official Documentation: [Managing Access Keys for IAM Users](#)
- 2. Rotate Credentials:**
 - Generate a new set of AWS access keys for the affected IAM user.
 - Update any applications, scripts, or services that use the old key with the new access key.
 - Store the new keys securely, using AWS Secrets Manager or a similar service to manage and access secrets safely.
- 3. Remove the Secret from Git History:**
 - Use **BFG Repo-Cleaner** or **git filter-repo** to remove the secret from the Git history:

Use case 3: Private Key Exposure

- AccuKnox detected exposed private key (.pem file) in the repository
- This may lead to unauthorized system access and full compromise
- AccuKnox proposes the solution to move the key, revoke and regenerate keys.

PrivateKey PLAIN Secret

High

Age	Severity	SLA	CVE ID	Tickets Created
2 days	High	45 days	NA	0

Description

Other Info

Raw Information

Detected a PrivateKey PLAIN data in the 'misc-keys/cert-key.pem' file within the Git repository 'https://github.com/udit-uniyal/leaky-repo', committed on 2025-08-29 11:16:37 +0000. The privatekey plain secret was found on line 1 and is associated with the user 'Udit Uniyal <115368361+udit-uniyal@users.noreply.github.com>'. The repository's commit ID is '876474d850649bbe9266a824817a557e5ab7d41a'.
[Show More](#)

 Finding for in resource

secret_git_Infura | secret_git_Infura

 First detected

on 28/08/2025

 Last detected

on 29/08/2025

Compliance Frameworks

No compliance found

Create Ticket

Ask AI

Asset

https://github.com/udit-uniyal/leaky-repo

Asset Type

secret_git_Infura

Status

Active

Ignored

No

Notes

Add Comments and Press Ctrl + Enter to Submit

Ask AI

Summarize the remediation steps for the exposed private key.

Immediate Remediation Steps

- Revoke the Exposed Key:**
 - If this key is associated with any service or application, immediately revoke or disable it. If applicable, use the relevant console or interface where the key is registered.
- Rotate Credentials:**
 - Generate a new private key securely. Ensure that this new key is stored in a secure location and not committed to the repository.
 - Update any systems or services that used the old key with the new key.
- Remove the Secret from Git History:**
 - Use `git filter-repo` or `BFG Repo-Cleaner` to remove the sensitive information from your Git history.

Using BFG Repo-Cleaner:

```
bash
bfg --delete-files cert-key.pem
git reflow expire --expire=now --all && git gc --prune=now --aggressive
```

Copy code

Using git filter-repo:

```
bash
```

Copy code

confidential and proprietary - limited distribution under NDA

46